SummerCal v2

Private iOS IPA Blueprint

AI Day Planner + Smart Notifications + Weather + Places + Money Tracker

Updated feature revision: Free-day notifications, weather alerts, upcoming-event alerts, and event notes

Date: 2026-05-23

Decision	Recommended choice
Distribution	Private Ad Hoc IPA installed on registered iPhone
Publishing	No App Store, no TestFlight, no public backend required for MVP
Core app	SwiftUI + SwiftData + UserNotifications
Smart features	AI provider router + WeatherKit + Core Location + MapKit / optional Google Places
New notification engine	Local notifications scheduled after precomputing calendar, weather, and AI suggestion context
Data privacy	Local-first storage, user-provided API keys stored in Keychain, clear permission prompts

Table of Contents

1. Product summary

SummerCal v2 is a private iOS-only calendar app distributed as a signed IPA for personal use. It should remain local-first, fast, and practical. The app combines a today-focused calendar, reminders, monthly income tracking, AI planning, weather context, location-aware suggestions, and nearby place discovery.

Core question	SummerCal answer
What do I have today?	Today dashboard, timeline, upcoming event card, and event reminders.
Is today free?	Free-day detector checks the calendar and sends optional activity suggestions.
What should I do?	AI Day Planner suggests realistic plans based on time, weather, location, open places, budget, and preferences.
What is the weather?	Weather notifications and daily weather summary based on WeatherKit or another provider.
Where can I go?	Nearby places and open-hours checks using MapKit first, Google Places if programmatic opening hours are required.
How much money did I make?	Monthly earnings calculator with manual income and work-session entries.

Product principle

The app should not feel like a bloated productivity system. It should answer today-specific questions quickly, then allow the user to go deeper only when needed.

2. Updated feature list

Feature	Description	MVP priority
Today calendar	Shows today, next event, current event, and timeline.	Must have
Event reminders	Local notifications before events.	Must have
Free-day notification	If a day has no events or has a large free window, notify with ideas.	Must have
AI activity ideas	Suggest activities from weather, location, open places, schedule, and preferences.	Must have
Weather notification	Morning summary and threshold alerts such as rain, heat, snow, or wind.	Must have

Upcoming event alert	Smart reminder with event title, notes preview, and preparation hints.	Must have
Event notes	Add notes, checklists, links, prep notes, and post-event notes.	Must have
In-app AI chat	Ask questions about the day, schedule, weather, nearby places, or money.	Should have
Nearby places	Find cafes, gyms, restaurants, parks, shops, or errands near current location.	Should have
Open hours	Show whether places are likely open now; use Google Places when precise programmatic hours are needed.	Should have
Money calculator	Monthly gross/net estimate, work-session calculator, goal progress.	Must have

3. Target architecture

Layer	Technology / module	Responsibility
UI	SwiftUI	Today, Calendar, Money, AI Planner, Places, Settings, Event Detail screens.
Local storage	SwiftData	Events, notes, income entries, work sessions, preferences, notification logs, suggestion cache.
Notifications	UserNotifications	Local reminders, free-day prompts, weather summaries, upcoming-event alerts.
Background refresh	BackgroundTasks	Best-effort refresh of weather, suggestions, and next-day planning cache.
Location	Core Location	Current city/coordinates for weather and places search.
Weather	WeatherKit preferred	Current conditions, hourly/daily forecast, rain/heat/wind logic.
Places	MapKit first; Google Places optional	Nearby places, place details, directions, business hours when available.
AI	Provider router	OpenAI, Claude, DeepSeek, and

		custom OpenAI-compatible endpoints.
Secrets	Keychain	Store user API keys securely on device.
Distribution	Ad Hoc IPA	Signed private IPA installed on registered iPhone.

```
SummerCal/
  App/
    SummerCalApp.swift
    AppRouter.swift

  Features/
    Today/
    Calendar/
    EventNotes/
    SmartNotifications/
    AIPlanner/
    Places/
    Weather/
    Money/
    Settings/

  Models/
    CalendarEvent.swift
    EventNote.swift
    EventReminder.swift
    SmartNotificationRule.swift
    NotificationLog.swift
    ActivitySuggestion.swift
    WeatherSnapshot.swift
    PlaceCandidate.swift
    IncomeEntry.swift
    WorkSession.swift
    UserSettings.swift

  Services/
    NotificationService.swift
    SmartNotificationService.swift
    FreeDayDetectionService.swift
    ActivitySuggestionService.swift
    AIProviderRouter.swift
    LocationService.swift
    WeatherService.swift
    PlacesService.swift
    EventNoteService.swift
    EarningsService.swift

  Storage/
    Repositories/
```

```
    KeychainStore.swift

  Utilities/
    DateUtils.swift
    CurrencyFormatter.swift
    PromptBuilder.swift
```

4. AI Day Planner

The AI Day Planner should create realistic plans from the user schedule, weather, location, nearby open places, activity preferences, budget, and energy level. It should not override the calendar; it should suggest optional plans.

Input	Example
Calendar context	No events today, or free between 13:00 and 18:00.
Weather context	Cloudy, 18 C, rain likely after 17:00.
Location context	Current city or approximate area only, unless precise location is enabled.
Place context	Nearby cafes, parks, gyms, shops, restaurants; open-now flag when available.
User preferences	Indoor/outdoor, low-cost, productive, social, relaxing, fitness, errands.
Money context	Optional: avoid expensive ideas if spending limit is low.

```
AIPlannerRequest
  date
  freeWindows
  eventsSummary
  weatherSummary
  locationSummary
  nearbyPlaces
  userPreferences
  budgetPreference
  energyLevel
  providerConfig

AIPlannerResponse
  daySummary
  suggestions[3-5]
  recommendedPlan
  notificationTitle
  notificationBody
  reasons
```

Provider router

The app should support multiple AI providers through one internal protocol. The user enters an API key in Settings and selects a provider. Keep all provider-specific payload logic outside the UI.

```
protocol AIProviderClient {
    var providerName: String { get }
    func sendChat(messages: [AIMessage], config: AIRequestConfig) async throws -> AIResponse
}

final class AIProviderRouter {
    func client(for provider: AIProviderKind) -> AIProviderClient {
        switch provider {
        case .openAI: return OpenAIClient()
        case .claude: return ClaudeClient()
        case .deepSeek: return DeepSeekClient()
        case .openAICompatible: return OpenAICompatibleClient()
        }
    }
}
```

5. Smart Notification Engine

The Smart Notification Engine decides what to notify, when to notify, and what content appears. It should avoid notification spam. The default should be useful and quiet.

Notification type	Trigger condition	Default timing
Upcoming event	Event starts soon and reminder is enabled.	15 minutes before event.
Free day	No events today, or largest free window exceeds the user threshold.	Morning, e.g. 09:00.
Free afternoon	No events after lunch, or free window from 13:00 onward.	Late morning, e.g. 11:00.
Weather summary	Daily weather notifications enabled.	Morning, e.g. 07:30.
Weather alert	Rain, heat, snow, wind, storm, or outdoor-event conflict threshold is met.	Before affected period.
Event prep	Event has notes/checklist items or AI can infer preparation suggestions.	30-60 minutes before event.
Money reminder	Monthly goal progress is behind or payday/income entry reminder is due.	Evening or user-configured time.

Design rule	Reason

Precompute notification content	Local notifications should already contain title/body by the time they are scheduled.
Do not call AI at notification fire time	iOS will not wake the app just because the notification is displayed.
Use local notifications for MVP	No backend or APNs server is needed for a private IPA.
Cache weather and place context	Prevents slow notifications and unnecessary API calls.
Throttle aggressively	A personal calendar app becomes annoying if it sends too many suggestions.

SmartNotificationPipeline

1. App opens or background refresh runs.
2. Load tomorrow and today events from SwiftData.
3. Detect free windows and upcoming events.
4. Fetch weather if enabled and permission/API setup exists.
5. Fetch nearby/open places only when suggestions need them.
6. Call selected AI provider only if AI suggestions are enabled.
7. Save SmartSuggestion and NotificationLog.
8. Schedule local notifications with fixed title/body/action IDs.

6. Free-day detection and activity ideas

A free day is not only a day with zero events. A day can be considered free if it has a large uninterrupted block of time. This should be configurable.

Setting	Recommended default
Notify if day has zero events	On
Notify if largest free window exceeds	4 hours
Free-day check time	09:00
Next-day free-day preview	Optional at 20:00 previous evening
Activity idea count	3 quick ideas + 1 recommended plan
Quiet hours	22:00 - 08:00
Maximum smart notifications	2 per day by default

```
func freeWindows(for date: Date, events: [CalendarEvent]) -> [DateInterval] {
    let day = DateUtils.dayInterval(date)
    let busy = events
        .filter { $0.overlaps(day) }
        .map { DateInterval(start: max($0.startDate, day.start), end: min($0.endDate, day.end)) }
        .sorted { $0.start < $1.start }

    return DateUtils.invertBusyIntervals(busy, within: day)
}

func shouldNotifyFreeDay(freeWindows: [DateInterval], thresholdHours: Double) -> Bool {
```

```
    freeWindows.contains { $0.duration >= thresholdHours * 3600 }
}
```

Notification example	Body
Free day detected	Your day is mostly free. Want 3 ideas based on the weather and what is open nearby?
Free afternoon	You have a 5-hour free window this afternoon. Good time for errands, gym, or a cafe work block.
Sunny free day	Clear weather today. Good options: walk, outdoor cafe, workout, or city exploration.
Rainy free day	Rain is likely later. Better ideas: indoor cafe, cinema, museum, gym, or focused work session.

7. Weather notifications

Weather notifications should make the calendar more useful. The app should notify about weather only when it affects the day or when the user explicitly enables daily summaries.

Weather notification	Trigger
Morning summary	Daily weather notifications enabled.
Rain alert	Precipitation probability exceeds threshold during active/free hours.
Outdoor event warning	Outdoor-tagged event overlaps rain, snow, wind, heat, or poor conditions.
Umbrella reminder	Rain likely before or during an event.
Heat/cold alert	Temperature crosses user-defined threshold.
Wind/storm alert	Strong wind or severe condition appears in forecast.

```
WeatherNotificationLogic
  if dailySummaryEnabled:
    schedule morning summary

  if rainProbability > user.rainThreshold during upcoming event window:
    schedule event weather warning

  if dayIsFree and weatherIsGood:
    include outdoor suggestions
  else if dayIsFree and weatherIsBad:
    include indoor suggestions
```

Weather data should be fetched and cached when the app opens and through best-effort background refresh. If background refresh does not run, the app should refresh weather the next time the user opens it and schedule future notifications then.

8. Upcoming-event notifications

Upcoming-event alerts should be more useful than simple time reminders. They should include notes, checklist hints, location/weather context, and preparation suggestions when available.

Reminder level	Example notification
Basic	Client call starts in 15 minutes.
With notes	Client call starts in 15 minutes. Notes: discuss landing page changes.
With checklist	Client call in 30 minutes. Prep: open proposal, review invoice, prepare questions.
With weather	Gym in 45 minutes. Rain starts around the same time - leave earlier or take an umbrella.
With AI prep	Meeting in 1 hour. Suggested prep: review last note, confirm goal, write 3 talking points.

Notification action	Behavior
Open event	Deep-links to EventDetailView.
Show notes	Opens notes/checklist tab for the event.
Snooze 10 min	Cancels and reschedules one reminder.
Mark prepared	Updates event prep status.
Plan around it	Opens AI Planner with that event as context.

9. Event notes and event memory

Events should support rich notes. For the MVP, use plain text notes plus optional checklists. This avoids overbuilding a full document editor while still making event reminders much more useful.

Note type	Purpose
General note	Freeform details about the event.
Prep note	What to do before the event.
Checklist	Small tasks that can be completed before or during the event.
Link list	URLs, meeting links, maps, documents, invoices.
Post-event note	Outcome, follow-up tasks, useful memory for future events.
AI summary	Optional generated summary from long notes or chat context.

```
@Model
final class EventNote {
    var id: UUID
    var eventId: UUID
```

```
    var body: String
    var noteType: EventNoteType
    var checklistItemsJSON: String?
    var linksJSON: String?
    var createdAt: Date
    var updatedAt: Date

    init(
        id: UUID = UUID(),
        eventId: UUID,
        body: String,
        noteType: EventNoteType = .general,
        checklistItemsJSON: String? = nil,
        linksJSON: String? = nil,
        createdAt: Date = Date(),
        updatedAt: Date = Date()
    ) {
        self.id = id
        self.eventId = eventId
        self.body = body
        self.noteType = noteType
        self.checklistItemsJSON = checklistItemsJSON
        self.linksJSON = linksJSON
        self.createdAt = createdAt
        self.updatedAt = updatedAt
    }
}
```

UI element	Behavior
Notes tab in Event Detail	Displays general notes, prep notes, checklist, and links.
Quick add note button	Adds a note from the event screen without opening a full editor.
Pin note to reminder	Allows a note snippet to appear in upcoming-event notification body.
AI summarize notes	Summarizes long notes into 1-3 reminder-friendly bullets.
Follow-up extraction	AI can suggest follow-up tasks from post-event notes.

10. Location, weather, places, and open hours

The AI Planner becomes useful when it can combine schedule with environment. Keep the flow permission-aware and graceful when permissions are denied.

Need	Recommended implementation
Get location	Core Location with When In Use permission. Use approximate location if possible.

Get weather	WeatherKit for native iOS. Optional fallback provider if WeatherKit is not configured.
Find places	MapKit local search for nearby cafes, parks, gyms, restaurants, shops, museums, errands.
Show place details	MapKit item detail UI when available.
Check open hours	Google Places API optional if you need reliable programmatic opening hours/open-now filtering.
Navigation	Open Apple Maps from selected place.

Important decision: MapKit is good for native Apple Maps integration and nearby points of interest. If the product requirement is to programmatically filter by exact opening hours or open-now state before showing AI suggestions, add a PlacesProvider abstraction and implement Google Places as an optional provider.

```
protocol PlacesProvider {
    func searchNearby(query: String, coordinate: Coordinate, radiusMeters: Double) async throws -> [PlaceCandidate]
    func details(placeId: String) async throws -> PlaceDetails
}

final class PlacesService {
    let mapKitProvider: PlacesProvider
    let googlePlacesProvider: PlacesProvider?

    func openNowCandidates(for query: String, location: Coordinate) async throws -> [PlaceCandidate] {
        if let googlePlacesProvider {
            return try await googlePlacesProvider.searchNearby(query: query, coordinate: location, radiusMeters: 3000)
                .filter { $0.openNow != false }
        }
        return try await mapKitProvider.searchNearby(query: query, coordinate: location, radiusMeters: 3000)
    }
}
```

11. Data model additions

SmartNotificationRule

```
@Model
final class SmartNotificationRule {
    var id: UUID
    var kind: SmartNotificationKind
    var isEnabled: Bool
    var preferredHour: Int
    var preferredMinute: Int
    var quietHoursStart: Int
    var quietHoursEnd: Int
    var maxPerDay: Int
    var createdAt: Date
    var updatedAt: Date
```

```
}
```

ActivitySuggestion

```
@Model
final class ActivitySuggestion {
    var id: UUID
    var date: Date
    var title: String
    var summary: String
    var category: String
    var estimatedDurationMinutes: Int
    var estimatedCostLevel: Int
    var placeName: String?
    var placeId: String?
    var weatherReason: String?
    var aiProvider: String?
    var createdAt: Date
}
```

NotificationLog

```
@Model
final class NotificationLog {
    var id: UUID
    var notificationId: String
    var kind: SmartNotificationKind
    var title: String
    var body: String
    var scheduledFor: Date
    var deliveredEstimate: Date?
    var relatedEventId: UUID?
    var relatedSuggestionId: UUID?
    var createdAt: Date
}
```

WeatherSnapshot

```
@Model
final class WeatherSnapshot {
    var id: UUID
    var latitude: Double
    var longitude: Double
    var fetchedAt: Date
    var forecastDate: Date
    var condition: String
    var temperatureCelsius: Double
    var precipitationChance: Double
    var windSpeedKph: Double?
    var summary: String
}
```

12. iOS permissions and entitlements

Capability / permission	Required for	MVP recommendation
Notifications	Event reminders, free-day alerts, weather alerts.	Required. Ask during onboarding after explaining value.
Location When In Use	Weather by current location, nearby places, local suggestions.	Optional but strongly recommended for AI Planner.
WeatherKit capability	Native Apple weather data.	Use if you have a paid developer account and want Apple weather data.
Background Modes / BackgroundTasks	Best-effort refresh of weather and suggestions.	Enable after MVP notification engine works.
Calendar access / EventKit	If importing/exporting Apple Calendar events.	Optional later. Use local calendar first.
Keychain	User-entered API keys.	Required for AI provider keys.

Info.plist key	Purpose
NSLocationWhenInUseUsageDescription	Explain location use for weather and nearby activity suggestions.
NSUserNotificationsUsageDescription is not an Info.plist key	Notifications require runtime permission through UserNotifications.
NSCalendarsFullAccessUsageDescription	Only if using EventKit full calendar access.
NSCalendarsWriteOnlyAccessUsageDescription	Only if writing events to Apple Calendar without needing to read existing calendar data.

13. Screens and UX

Screen	New elements
Today	Weather strip, next event, free-day banner, activity suggestions, money card.
Calendar	Month/day agenda with free windows highlighted.
Event Detail	Notes tab, prep checklist, links, weather near event time, AI prep button.
AI Planner	Prompt shortcuts: Plan my free day, Find something nearby, What should I do before my event?
Places	Search nearby, category filters, open-now indicator, route button.
Weather	Today summary, hourly forecast, alert thresholds.
Notifications Settings	Enable/disable smart alerts, quiet hours, free-day

	threshold, weather alerts.
AI Settings	Provider selection, API key fields, model field, base URL for compatible APIs.

UX copy	Suggested text
Notification permission explainer	SummerCal can remind you before events, warn you about weather, and suggest ideas when your day is free.
Location permission explainer	Use your location to show local weather and nearby places. You can disable this anytime.
Free-day banner	Your day is mostly free. Want a plan based on weather and places nearby?
Event notes placeholder	Add notes, checklist items, links, or anything you need before this event.
Weather alert setting	Notify me when weather may affect my events or free-day plans.

14. Implementation roadmap

Phase	Build scope
1. Core local app	SwiftUI tabs, SwiftData models, Today screen, add/edit/delete events, event notes, income entries.
2. Basic notifications	Notification permission, event reminders, local scheduling, cancel/reschedule logic.
3. Smart notifications	Free-day detector, notification settings, quiet hours, notification log, daily scheduling pipeline.
4. Weather module	Location permission, WeatherKit integration, weather cache, daily summary, weather thresholds.
5. AI Planner	Provider router, Keychain API keys, in-app chat, planning prompts, suggestion cache.
6. Places module	MapKit local search, place cards, route open in Maps, optional Google Places open-hours provider.
7. Polish and packaging	QA, settings, edge cases, export signed Ad Hoc IPA.

MVP cut line

- Must ship first: events, event notes, event reminders, Today screen, Money screen, local storage.
- Then add: free-day notifications, weather summary, AI suggestions.
- Then add: open-now places and richer AI place-based plans.

15. QA checklist

Area	Test cases
Event reminders	Create, edit, delete event; reminder schedules, reschedules, and cancels correctly.
Free-day notification	Zero events day, partially free day, busy day, quiet hours, max-per-day limit.
Weather alerts	Rain threshold, outdoor event overlap, daily summary on/off, permission denied fallback.
Event notes	Add/edit/delete notes, checklist completion, notes preview in notification, long text handling.
AI providers	OpenAI, Claude, DeepSeek, custom endpoint; invalid API key; timeout; offline mode.
Location/places	Permission granted, denied, approximate location, no results, place opening hours missing.
Private IPA	Archive, export Ad Hoc IPA, install on registered iPhone, notification permissions after install.

- Test on a real iPhone, not only simulator, because notifications, location, WeatherKit, and background refresh behave differently on real devices.
- Use NotificationLog records to debug scheduled notifications.
- Avoid relying on exact background refresh timing. iOS controls when background tasks run.
- Always provide fallbacks: no location, no weather, no AI key, no open-hours data.

16. Private IPA signing and installation

Because the app will not be published to the App Store, the recommended distribution path remains Ad Hoc IPA distribution through a paid Apple Developer account.

1. Create the Xcode project with bundle identifier such as com.yourname.summercalv2.
2. Add your Apple Developer account in Xcode.
3. Register your iPhone UDID in Apple Developer Certificates, Identifiers & Profiles.
4. Create an App ID and enable required capabilities such as WeatherKit and Background Modes if used.
5. Create or select an Apple Distribution certificate.
6. Create an Ad Hoc provisioning profile for the app and selected iPhone.
7. Archive the app in Xcode.
8. Export as Ad Hoc IPA.
9. Install using Apple Configurator or Xcode Devices window.

For a private personal app, do not add a backend unless you need cross-device sync, shared calendars, server-side AI key proxying, or push notifications. For this blueprint, local notifications are enough.

17. Reference docs

Topic	URL
Apple UserNotifications	https://developer.apple.com/documentation/usernotifications
Scheduling local notifications	https://developer.apple.com/documentation/usernotifications/scheduling-a-notification-locally-from-your-app
Apple BackgroundTasks	https://developer.apple.com/documentation/backgroundtasks
Background app refresh sample	https://docs.developer.apple.com/documentation/BackgroundTasks/refreshing-and-maintaining-your-app-using-background-tasks
Apple Core Location	https://developer.apple.com/documentation/corelocation
Apple WeatherKit REST API	https://developer.apple.com/documentation/weatherkitrestapi
Apple MapKit	https://developer.apple.com/documentation/mapkit
MapKit nearby points of interest	https://developer.apple.com/documentation/mapkit/interacting-with-nearby-points-of-interest
Google Places API overview	https://developers.google.com/maps/documentation/places/web-service/overview
Google Place Details	https://developers.google.com/maps/documentation/places/web-service/place-details
Apple Ad Hoc provisioning profile	https://developer.apple.com/help/account/provisioning-profiles/create-an-ad-hoc-provisioning-profile
Xcode distribution to registered devices	https://help.apple.com/xcode/mac/current/en.lproj/dev7ccaf4d3c.html